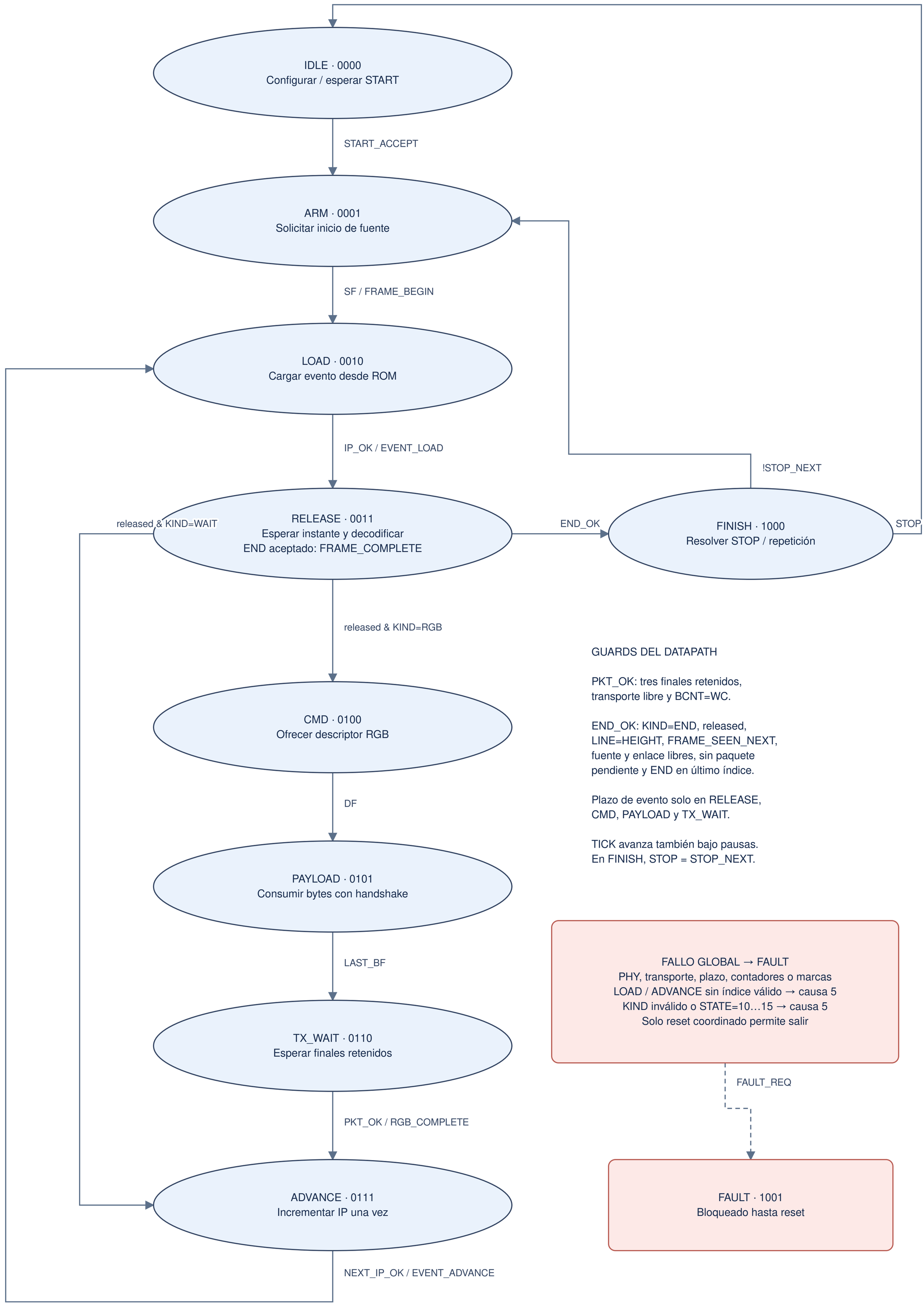
FSM · PLANIFICADOR DSI · TRANSICIONES

PROPUESTA · STATE[3:0], diez códigos usados. Registros en ↑clk; rst síncrono activo alto.
Toda transición normal exige !FAULT_REQ. Los controles de handshake son combinacionales antes del flanco.



RETENCIÓN Y PRIORIDAD
Sin guard habilitado, conservar STATE. LOAD, ADVANCE y FINISH resuelven en un ciclo sin fallo.
1. rst; 2. FAULT_REQ; 3. transición normal. FAULT es absorbente; STOP no aborta el cuadro.
SF = source.start_valid & source.start_ready. DF = cmd_valid & cmd_ready. LAST_BF = BF & (BCNT=WC−1).

FSM · CONTROLES Y CONDICIONES DE SALIDA

Todos los pulsos son enables presentes antes del flanco que ejecuta la operación; no registros ACTION.
Señales no indicadas = 0. La captura de eventos y los contadores siguen las ecuaciones del datapath.

ESTADO [4]	CONTROLES ACTIVOS / ENTRADAS AL DATAPATH	
IDLE 0000	stopped = chain_free & !rst & !R_ERROR.valid; habilita CSR	START_ACCEPT → ARM; si no, conservar estado
ARM 0001	source.start_valid = !HARD_BLOCK	SF → LOAD; sin ready, conservar estado
LOAD 0010	EVENT_LOAD = IP_OK & !FAULT_REQ	IP_OK → RELEASE; si no, FAULT / causa 5
RELEASE 0011	Según KIND y released; END puede emitir FRAME_COMPLETE	WAIT → ADVANCE; RGB → CMD; END_OK → FINISH
CMD 0100	cmd_valid = !HARD_BLOCK; campos desde R_ACTIVE	DF → PAYLOAD; sin ready, conservar descriptor
PAYLOAD 0101	G = !HARD_BLOCK & (BCNT < WC)	LAST_BF → TX_WAIT; sin BF, conservar byte y contadores
TX_WAIT 0110	Capturar finales; RGB_COMPLETE = PKT_OK & !FAULT_REQ	PKT_OK → ADVANCE; si no, conservar estado
ADVANCE 0111	EVENT_ADVANCE = NEXT_IP_OK & !FAULT_REQ	NEXT_IP_OK → LOAD; si no, FAULT / causa 5
FINISH 1000	Cuadro ya contado; frame_active = 0	STOP_NEXT → IDLE; si no, ARM para repetir
FAULT 1001	running=0; stopped=0; bloquear nuevos inicios	Conservar estado hasta rst; no hay CLEAR_ERROR parcial

MÁSCARAS Y VENTANAS
HARD_BLOCK = rst | estado ilegal | STATE=FAULT | R_ERROR.valid | fallo urgente sin depender de DF/BF.
cmd_valid y G usan HARD_BLOCK; mark_error / count_error derivados de BF no vuelven a enmascarar ese mismo BF.
frame_active: LOAD / RELEASE / CMD / PAYLOAD / TX_WAIT / ADVANCE.
pkt_window: DF o PAYLOAD / TX_WAIT; frame_window: FRAME_BEGIN o frame_active.
STOP se captura mientras running o START_ACCEPT. running: todos excepto IDLE / FAULT. stopped: IDLE & chain_free.

MEMORIA UC: STATE[4] + config_ok[1] + panel_initialized[1] = 6 bits. Ventanas y flags de actividad se decodifican de STATE.
Total propuesto: datapath 198 + CSR R_SHADOW 10 + UC 6 = 214 bits, sin ROM ni módulos externos.
Comandos detenidos: cfg_write > APPLY > escritura de autorización > START. STOP puede coincidir con START.
Diseño documentado, no RTL implementado. Verificación simbólica: verificar_fsm.py; límites y ecuaciones: fsm.md.